Test Author

G3_Apache_Spark_Report.pdf

Ho Technical University

Document Details

Submission ID
trn:oid:::1:769256586

Submission Date
May 18, 2026, 8:24 PM GMT+5

Download Date
May 18, 2026, 8:26 PM GMT+5

File Name
G3_Apache_Spark_Report.pdf

File Size
2.2 MB

33 Pages

3,852 Words

25,038 Characters

30% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

47 AI-generated only 30%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 1

APACHE SPARK TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 3

STUDENT NAME	MATRIC NO
ELIJAH SHE YU SHENG	A23CS0073
CHERYL CHEONG KAH VOON	A23CS0060
CHUA JIA LIN	A23CS0069

TABLE OF CONTENTS

1.0 BUSINESS UNDERSTANDING

The purpose of this project is to build an Apache Spark-based ETL pipeline for processing large datasets, also known as Censo Escolar. The dataset contains yearly school census records from 2010 to 2021 which consist of information related to school location, administrative dependency, infrastructure facilities, internet availability, computer availability, enrolment counts, teacher counts, and class statistics.

The main business problem is large raw data, fragmented across multiple yearly CSV files, and not directly suitable for efficient analytical reporting. Each yearly dataset contains many records and attributes, making it difficult to analyze using traditional spreadsheet-based tools. Without proper data engineering workflow, the analysts may face challenges such as combining yearly data, cleaning inconsistent values, transforming raw files, and preparing the data for creating dashboard reports.

Therefore, Apache Spark is the main processing tool because it supports scalable and distributed data processing. In this project, Spark was used to transform raw CSV files into Parquet format to generate dimension tables, create a fact table, and load the structured data into PostgreSQL. PostgreSQL acts as the data warehouse, while Metabases is used for dashboard visualization and analytical reporting.

Besides, the business objective of this project is to convert raw data into a structured star schema model that supports efficient analysis. This allows users to explore education-related trends such as student enrolment, teacher availability, school infrastructure, internet access, and school distribution across regions and years. The expected users of this system such as education analysts and school administrators need reliable insights for planning and decision-making.

Overall, this project demonstrates how Apache Spark can be applied in a data engineering workflow to process large datasets and transform them into analysis-ready data for business intelligence reporting.

2.0 DATA UNDERSTANDING

The dataset used in this project is the Censo Escolar dataset from the Brazilian School Census. It consists of 12 yearly CSV files from 2010 to 2021. These files were stored in the "/data" directory after the dataset extraction process. Each yearly file follows a similar schema, allowing the datasets to be integrated into a unified dataset for Spark-based processing.

The dataset contains many records with a wide range of attributes. The major groups of attributes such as census year, school identification, geographical information, administrative dependency, school location, infrastructure availability, utility access, technology availability, enrolment statistics, teacher counts, and class counts. These attributes are very useful for understanding the condition and development of schools across different years and regions.

The important attributes include NU_ANO_CENSO, which represents the census year; CO_ENTIDADE and NO_ENTIDADE which identify the school entity; NO_REGIAO, NO_UF, SG_UF, and NO_MUNICIPIO which describe the geographical location of the school; TP_DEPENDENCIA which represents the school administrative dependency; and TP_LOCALIZACAO which describes whether the school is located in an urban or rural area. Other than that, the infrastructure-related attributes such as IN_AGUA_POTAVEL, IN_BIBLIOTECA, IN_INTERNET, IN_COMPUTADOR, and IN_BANHEIRO. Numerical measures include enrolment, teacher, and class-related columns such as QT_MAT_BAS, QT_DOC_BAS, and QT_TUR_BAS.

During the data understanding, it was identified that the dataset is large and contains millions of records. In the Apache Spark execution, the Parquet dataset contained 2,792,984 rows. This ensured that the dataset is suitable for demonstrating the use of Apache Spark in large-scale data processing. At the same time, the dataset also contains incomplete records and null values which may affect the analysis accuracy if the datasets are not handled properly.

The dataset was transformed into a relational and multidimensional structure. For relational understanding, the main dataset can be represented as a school census record table, where each record describes a school in a specific census year. The data can be further normalized into separate entities such as school, location, infrastructure, and census facts.

2.1 RELATIONAL DATA MODEL

The relational data model represents the logical structure of the Censo Escolar dataset before it is transformed into the multidimensional star schema. The dataset can be organized into several related tables, including School, Location, Infrastructure, and Census Record. This structure helps separate descriptive school information, geographical details, infrastructure attributes, and yearly numerical census measures.

Table Name	Attribute	Key Type	Description
School	CO_ENTIDADE	Primary Key	Unique school entity code.
	NO_ENTIDADE	Attribute	Name of the school.
	TP_DEPENDENCIA	Attribute / FK Candidate	Type of school administrative dependency, such as federal, state, municipal, or private.
	TP_LOCALIZACAO	Attribute / FK Candidate	Indicates whether the school is located in an urban or rural area.
Location	CO_MUNICIPIO	Primary Key	Unique municipality code.
	NO_MUNICIPIO	Attribute	Name of the municipality.
	CO_UF	Attribute	State code.
	SG_UF	Attribute	State abbreviation.
	NO_UF	Attribute	Name of the state.
	CO_REGIAO	Attribute	Region code.
	NO_REGIAO	Attribute	Name of the region.
Infrastructure	CO_ENTIDADE	Foreign Key	Refers to the school entity in the School table.
	IN_AGUA_POTAVEL	Attribute	Indicates whether the school has potable water.
	IN_ENERGIA_INEXISTENTE	Attribute	Indicates whether the school has no electricity access.
	IN_ESGOTO_INEXISTENTE	Attribute	Indicates whether the school has no sewage system.
	IN_BANHEIRO	Attribute	Indicates whether the school has bathroom facilities.
	IN_BIBLIOTECA	Attribute	Indicates whether the school has a library.
	IN_REFEITORIO	Attribute	Indicates whether the school has a cafeteria or dining area.
	IN_COMPUTADOR	Attribute	Indicates whether the school has computers.
	IN_INTERNET	Attribute	Indicates whether the school has internet access.
	IN_EQUIP_NENHUM	Attribute	Indicates whether the school has no listed equipment.
Census_Record	NU_ANO_CENSO	Composite Key / Attribute	Year of the school census record.
	CO_ENTIDADE	Foreign Key	Refers to the school entity in the School table.
	CO_MUNICIPIO	Foreign Key	Refers to the municipality in the Location table.
	QT_MAT_BAS	Measure	Number of basic education enrolments.
	QT_MAT_INF	Measure	Number of early childhood education enrolments.
	QT_MAT_FUND	Measure	Number of elementary education enrolments.
	QT_MAT_MED	Measure	Number of secondary education enrolments.
	QT_DOC_BAS	Measure	Number of basic education teachers.
	QT_DOC_INF	Measure	Number of early childhood education teachers.
	QT_DOC_FUND	Measure	Number of elementary education teachers.
	QT_DOC_MED	Measure	Number of secondary education teachers.

	QT_TUR_BAS	Measure	Number of basic education classes.
	QT_TUR_INF	Measure	Number of early childhood education classes.
	QT_TUR_FUND	Measure	Number of elementary education classes.
	QT_TUR_MED	Measure	Number of secondary education classes.

In this relational model, the "School" table stores school identity and school type information, while the "Location" table stores geographical details such as region, state, and municipality. The "Infrastructure" table stores facility-related attributes for each school. The "Census Record" table stores yearly numerical measures such as student enrolment, teacher count, and class count. This relational structure helps explain the original dataset before it is transformed into the star schema used for multidimensional analysis.

3.0 DATA PREPARATION

The data preparation phase is an important step in the CRISP-DM process that emphasizes raw data transformation into clean, structured, and analysis-ready formats. In this project, data preparation was performed using Apache Spark to ensure scalability and efficient handling of large-scale Censo Escolar datasets from the Brazilian School Census.

3.1 DATASET EXTRACTION

The dataset used in this tutorial was the Censo Escolar datasets from Brazilian School Census. The Censo Escolar datasets were downloaded from the Brazilian Government's Open Data Portal using a Python script called *download_censo_escolar.py*.

The script:

- downloaded compressed files from year 2010 to year 2021
- extracted compressed files for each year
- identified CSV data files for each year
- stored all yearly CSV data files into */data* directory
- deleted remaining files

Figure 3.1.1 shows the code snippet of extracting censo escolar data files and store the data files in the */data* directory.

```python
YEARS = [str(i) for i in range(2010, 2022)]

BASE_URL = "https://download.inep.gov.br/dados_abertos/microdados_censo_escolar_{}.zip"

DATA_DIR = "./data"

os.makedirs(DATA_DIR, exist_ok=True)


def download_file(url, path, retries=5):
    for attempt in range(retries):
        try:
            print(f"Downloading {url} (attempt {attempt+1})")

            with requests.get(url, stream=True, timeout=60, verify=False) as r:
                r.raise_for_status()

                with open(path, "wb") as f:
                    for chunk in r.iter_content(chunk_size=1024 * 1024):  # 1MB chunks
                        if chunk:
                            f.write(chunk)

            print("Download complete ✓")
            return

        except Exception as e:
            print(f"Failed attempt {attempt+1}: {e}")
            time.sleep(5)

    raise Exception("Download failed after multiple retries")


for year in YEARS:
    zip_path = os.path.join(DATA_DIR, f"{year}.zip")
    extract_path = os.path.join(DATA_DIR, year)

    url = BASE_URL.format(year)

    # 1. Download
    if os.path.exists(zip_path):
        print(f"{zip_path} already exists, skipping download")
        continue

    download_file(url, zip_path)

    # 2. Extract
    print(f"Extracting {zip_path}")
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(extract_path)

    # 3. Find CSV file
    csv_file = None
    for root, dirs, files in os.walk(extract_path):
        for file in files:
            if file.lower().endswith(".csv"):
                csv_file = os.path.join(root, file)
                break

    # 4. Move CSV to main folder
    if csv_file:
        final_path = os.path.join(DATA_DIR, f"{year}.csv")
        shutil.move(csv_file, final_path)
        print(f"Saved: {final_path}")

    # 5. Cleanup
    shutil.rmtree(extract_path)
    os.remove(zip_path)

print("DONE ✓")
```

Figure 3.1.1 Code Snippet of Extracting Censo Escolar Datasets

Figure 3.1.2 shows that 12 CSV files were stored in the */data* directory after the execution of the *download_censo_escolar.py* script.

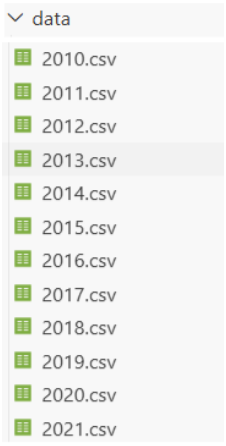

Figure 3.1.2 Result of *download_censo_escolar.py* Script

3.2 APACHE SPARK ENVIRONMENT SETUP

The census escolar datasets used in this tutorial contain millions of records. Therefore, Apache Spark was used to efficiently process large-scale distributed data, providing a scalable and high-performance solution for implementing the ETL pipeline.

The Apache Spark environment was configured using PySpark, which is the Python Application Programming Interface (API) for Apache Spark. PySpark allows developers to perform distributed data processing using Python syntax while utilizing Spark's parallel execution engine.

The Spark environment was initialized through the following steps:

- Imported *SparkSession* from *pyspark.sql* library
- Created a *SparkSession* object (*spark*) that:
  - defines the application name as *CensoEscolarStarSchema*
  - sets 4 partitions to be used during shuffle operations
  - initializes a new Spark session or retrieves an existing one if available

Figure 3.2.1 shows the code snippet of initializing a *SparkSession*.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder\
        .appName("CensoEscolarStarSchema")\
        .config("spark.sql.shuffle.partitions", "4")\
        .getOrCreate()
```

Figure 3.2.1 Code Snippet to Initialize Spark Environment

3.3 DATA TRANSFORMATION TO PARQUET FORMAT

After configuring the Apache Spark environment, the next step was transforming raw CSV files into Parquet, which is a columnar storage format to optimize big data processing.

During the transformation from CSV format to Parquet format:

- All CSV files were read using the *SparkSession* (*spark*)
- The data files were imported into a single distributed Spark DataFrame (*data_csv*) using .load() function
- The *data_csv* dataframe was saved in Parquet format using .write() function
- The data files in Parquet format were read using the *SparkSession* (*spark*)

Figure 3.3.1 shows the code snippet of transforming CSV data files into Parquet format.

```
# Read CSV data
data_csv = (
    spark
    .read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .option("delimiter", ";")
    .option("encoding", "iso-8859-1")
    .load([
        "/app/data/2010.csv",
        "/app/data/2011.csv",
        "/app/data/2012.csv",
        "/app/data/2013.csv",
        "/app/data/2014.csv",
        "/app/data/2015.csv",
        "/app/data/2016.csv",
        "/app/data/2017.csv",
        "/app/data/2018.csv",
        "/app/data/2019.csv",
        "/app/data/2020.csv",
        "/app/data/2021.csv"])
)

data_csv.write.mode("overwrite").parquet("/app/data/censo_escolar.parquet")

data = spark.read.parquet(
    "/app/data/censo_escolar.parquet"
)
```

Figure 3.3.1 Code Snippet of Transforming CSV to Parquet

3.4 POSTGRESQL CONFIGURATION

PostgreSQL was used as the relational database management system (RDBMS) for storing the transformed census escolar data. The database acts as a central data warehouse in the ETL pipeline, enabling efficient storage and multidimensional analysis of the processed datasets.

PostgreSQL integrates effectively with Apache Spark through JDBC (Java Database Connectivity). The integration allows Spark DataFrames to be written directly into relational tables for analytical processing.

The PostgreSQL connection parameters were configured as follows:

- *PG_HOST = "postgres"*
- *PG_PORT = 5432*
- *PG_DB = "censo_escolar"*
- *PG_USER = "censo"*
- *PG_PASSWORD = "123"*
- *PG_URL = f"jdbc:postgresql://{PG_HOST}:{PG_PORT}/{PG_DB}"*

Besides, a PostgreSQL configuration dictionary was also created to simplify database operations throughout the ETL process. Figure 3.4.1 shows the PostgreSQL configuration dictionary.

```
POSTGRES_CONFIG = {
    "url": PG_URL,
    "properties": {
        "user": PG_USER,
        "password": PG_PASSWORD,
        "driver": "org.postgresql.Driver"
    }
}
```

Figure 3.4.1 Code Snippet of PostgreSQL Configuration Dictionary

Moreover, a function (*get_connection*) was defined to establish a connection between a Python application and a PostgreSQL database using the psycopg2 library. The connection is used to execute SQL commands such as creating tables, inserting data, and managing constraints. Figure 3.4.2 shows the definition of *get_connection* function.

```
import psycopg2
conn = get_connection()
def get_connection():
    return psycopg2.connect(
        host=PG_HOST,
        port=PG_PORT,
        database=PG_DB,
        user=PG_USER,
        password=PG_PASSWORD
    )
```

Figure 3.4.2 Code Snippet for the *get_connection Function*

3.5 DIMENSION TABLE CREATION

After transforming the censo escolar data into Parquet format, the next stage of the ETL process involved the creation of dimension tables. Dimension tables are essential components in a star schema architecture because they store descriptive attributes used for data analytics.

In this tutorial, Apache Spark and PostgreSQL were used together to generate and store dimension tables. Spark handled the distributed transformation process, while PostgreSQL served as the data warehouse.

The dimension tables were created through the following steps:

- Defined 11 integer-based dimensions
- Configured a location dimension table (*DIM_LOCAL*)
- Dynamically configured dimension tables for 11 integer-based dimensions defined previously using Python dictionary comprehension (*for dimension in INTEGER_DIMENSIONS*)
- Performed transformation for every dimension table:
  - Selected relevant columns using .select() function
  - Cast columns into appropriate data types using .cast() function
  - Removed duplicate values using .distinct() function
  - Generate surrogate keys using .monotonically_increasing_id() function
- Wrote the dimension tables into PostgreSQL using JDBC write operations
- Added primary key constraints for every dimension table using psycopg2 library

Figure 3.5.1 and Figure 3.5.2 shows the code snippets of creating dimension tables.

```
# ------------------------------------------------
# Dimension tables
# ------------------------------------------------
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM"
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL": {
        "fields": [
            {"field": "NO_UF", "type": "string"},
            {"field": "SG_UF", "type": "string"},
            {"field": "CO_UF", "type": "string"},
            {"field": "NO_MUNICIPIO", "type": "string"},
            {"field": "CO_MUNICIPIO", "type": "string"}
        ]
    }
}

DIMENSION_TABLES_CONFIG.update(
    {
        "DIM_" + dimension.upper(): {
            "fields": [
                {"field": dimension, "type": "integer"}
            ]
        }
        for dimension in INTEGER_DIMENSIONS
    }
)
```

Figure 3.5.1 Code Snippet of Configuring Dimension Tables

```python
# Write dimensions
# ------------------------------------------------
for table_name, table_config in DIMENSION_TABLES_CONFIG.items():

    print(f"[{datetime.now()}] Writing {table_name}")

    dimension_df = (
        data
        .select(
            [
                F.col(field["field"])
                .cast(field["type"])
                .alias(field["field"])
                for field in table_config["fields"]
            ]
        )
        .distinct()
        .withColumn("id", F.monotonically_increasing_id())
    )

    dimension_df.write.jdbc(
        url=POSTGRES_CONFIG["url"],
        table=table_name,
        mode="overwrite",
        properties=POSTGRES_CONFIG["properties"]
    )

    print(f"[{datetime.now()}] Wrote {table_name}")

    # Add primary key
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
            f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);"
        )
        conn.commit()

    except Exception as e:
        print(f"PK already exists or error: {e}")
        conn.rollback()

    finally:
        cursor.close()
        conn.close()

    print(f"[{datetime.now()}] Done")
```

Figure 3.5.2 Code Snippet of Transforming and Creating Dimension Tables

3.6 FACT TABLE CREATION

After the generation of dimension tables, the ETL process proceeded with the creation of the fact table. In a star schema architecture, the fact table is the central table that stores measurable numerical data and references dimension tables through foreign keys.

The fact table created in this project was named *FACT_CENSO_ESCOLAR*.

The fact table was created through the following steps:

- Defined 15 fact columns (metrics) for the fact table
- Dynamically configured integer data type for 15 fact columns defined previously using Python dictionary comprehension (*for fact in FACT_COLUMNS*)
- Extracted dimension table fields to join fact data with dimension tables
- Added new fact columns using the extracted dimension table fields
- Created fact table using SQL commands through psycopg2:
  - Defined *id* as primary key
  - Stored fact columns in INTEGER data type
  - Stored dimension reference in BIGINT data type
- Added foreign key constraints to establish referential integrity between dimensions and facts
- Extracted the required fact columns and dimension attributes from the parquet dataset for joining
- Loaded all dimension tables from PostgreSQL and joined the dimension tables with the fact dataset to retrieve surrogate keys
- Replaced fact table's descriptive dimension attributes with foreign key references
- Inserted final fact dataset into PostgreSQL

Figure 3.6.1, Figure 3.6.2, Figure 3.6.3, and Figure 3.6.4 show the code snippets of creating fact table.

```python
# ------------------------------------------------
# Facts table
# ------------------------------------------------
print("Creating facts table...")

FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"

FACT_COLUMNS = [
    "QT_DOC_BAS",
    "QT_DOC_INF",
    "QT_DOC_FUND",
    "QT_DOC_MED",
    "QT_MAT_BAS",
    "QT_MAT_INF",
    "QT_MAT_FUND",
    "QT_MAT_MED",
    "QT_MAT_BAS_ND",
    "QT_MAT_BAS_BRANCA",
    "QT_MAT_BAS_PRETA",
    "QT_MAT_BAS_PARDA",
    "QT_MAT_BAS_AMARELA",
    "QT_MAT_BAS_INDIGENA",
    "NU_ANO_CENSO"
]

FACT_CONFIG = {
    fact: {
        "fields": [
            {"field": fact, "type": "integer"}
        ]
    }
    for fact in FACT_COLUMNS
}

DIMENSION_ID_CONFIG = {
    table_name: [
        field["field"]
        for field in table_fields["fields"]
    ]
    for table_name, table_fields in DIMENSION_TABLES_CONFIG.items()
}

FACT_TABLE_ALL_COLUMNS_ORDERED = (
    FACT_COLUMNS
    + list(map(lambda col: "ID_" + col, DIMENSION_ID_CONFIG.keys()))
)
```

Figure 3.6.1 Code Snippet of Configuring Fact Columns

```
# ------------------------------------------------
# Create facts table
# ------------------------------------------------
comma_break_line = ",\n"

conn = get_connection()
cursor = conn.cursor()

try:

    # ------------------------------------------------
    # Create FACT table
    # ------------------------------------------------
    facts_table_sql = f"""
    CREATE TABLE IF NOT EXISTS {FACT_TABLE_NAME} (
        id SERIAL PRIMARY KEY,
        {
            comma_break_line.join(
                [f"{field} INTEGER" for field in FACT_COLUMNS]
                +
                [
                    f"ID_{dim_table} BIGINT"
                    for dim_table in DIMENSION_ID_CONFIG.keys()
                ]
            )
        }
    );
    """

    cursor.execute(facts_table_sql)
    conn.commit()

    print(f"[{datetime.now()}] Created facts table")
```

Figure 3.6.2 Code Snippet of Creating Fact Table

```
        # ------------------------------------------------
        # Add foreign keys
        # ------------------------------------------------
        for dim_table in DIMENSION_ID_CONFIG.keys():

            constraint_name = f"fk_{FACT_TABLE_NAME}_{dim_table}"

            fk_sql = f"""
            ALTER TABLE {FACT_TABLE_NAME}
            ADD CONSTRAINT {constraint_name}
            FOREIGN KEY (ID_{dim_table})
            REFERENCES {dim_table}(id);
            """

            try:

                cursor.execute(fk_sql)
                conn.commit()

                print(
                    f"[{datetime.now()}] Added FK: "
                    f"{FACT_TABLE_NAME} -> {dim_table}"
                )

            except Exception as fk_error:

                print(
                    f"[{datetime.now()}] FK already exists "
                    f"or error: {fk_error}"
                )

                conn.rollback()

    except Exception as e:

        print(f"ERROR creating fact table: {e}")
        conn.rollback()

    finally:

        cursor.close()
        conn.close()

    # ------------------------------------------------
    # Prepare facts data
    # ------------------------------------------------
    facts_data = data.select(
        [
            *chain(
                *DIMENSION_ID_CONFIG.values(),
                FACT_CONFIG.keys()
            )
        ]
    )
```

Figure 3.6.3 Code Snippet of Adding Foreign Key Constraints in Fact Table

```
# ------------------------------------------------
# Join dimensions
# ------------------------------------------------
for table_name, table_fields in DIMENSION_ID_CONFIG.items():

    dim_table = (
        spark.read.jdbc(
            url=POSTGRES_CONFIG["url"],
            table=table_name,
            properties=POSTGRES_CONFIG["properties"]
        )
        .withColumnRenamed("id", f"ID_{table_name}")
    )

    facts_data = (
        facts_data
        .join(
            dim_table,
            on=table_fields,
            how="left"
        )
        .drop(*table_fields)
    )

# ------------------------------------------------
# Write facts table
# ------------------------------------------------
try:

    (
        facts_data
        .select(*FACT_TABLE_ALL_COLUMNS_ORDERED)
        .write
        .jdbc(
            url=POSTGRES_CONFIG["url"],
            table=FACT_TABLE_NAME,
            mode="append",
            properties=POSTGRES_CONFIG["properties"]
        )
    )

    print("Facts table written successfully")

except Exception as e:
    print(f"ERROR in facts table: {e}")
```

Figure 3.6.4 Code Snippet of Joining Dimension Tables with Fact Table and Write Fact Table to PostgreSQL

4.0 MODELLING

4.1 Star Schema

The multi-dimensional model is designed with a star architecture to support efficient analytical processing and business intelligence reporting. The star schema is shown in Figure 4.1. There is one central fact table (fact_censo_escolar), and several dimensional tables are connected with fact tables to provide descriptive information and analytical views. The fact table stores the main quantitative measures, such as the number of students, enrolments, and educational statistics collected from the dataset. This design improves query performance, simplifies data analysis, and supports more effective decision-making through multidimensional analysis.

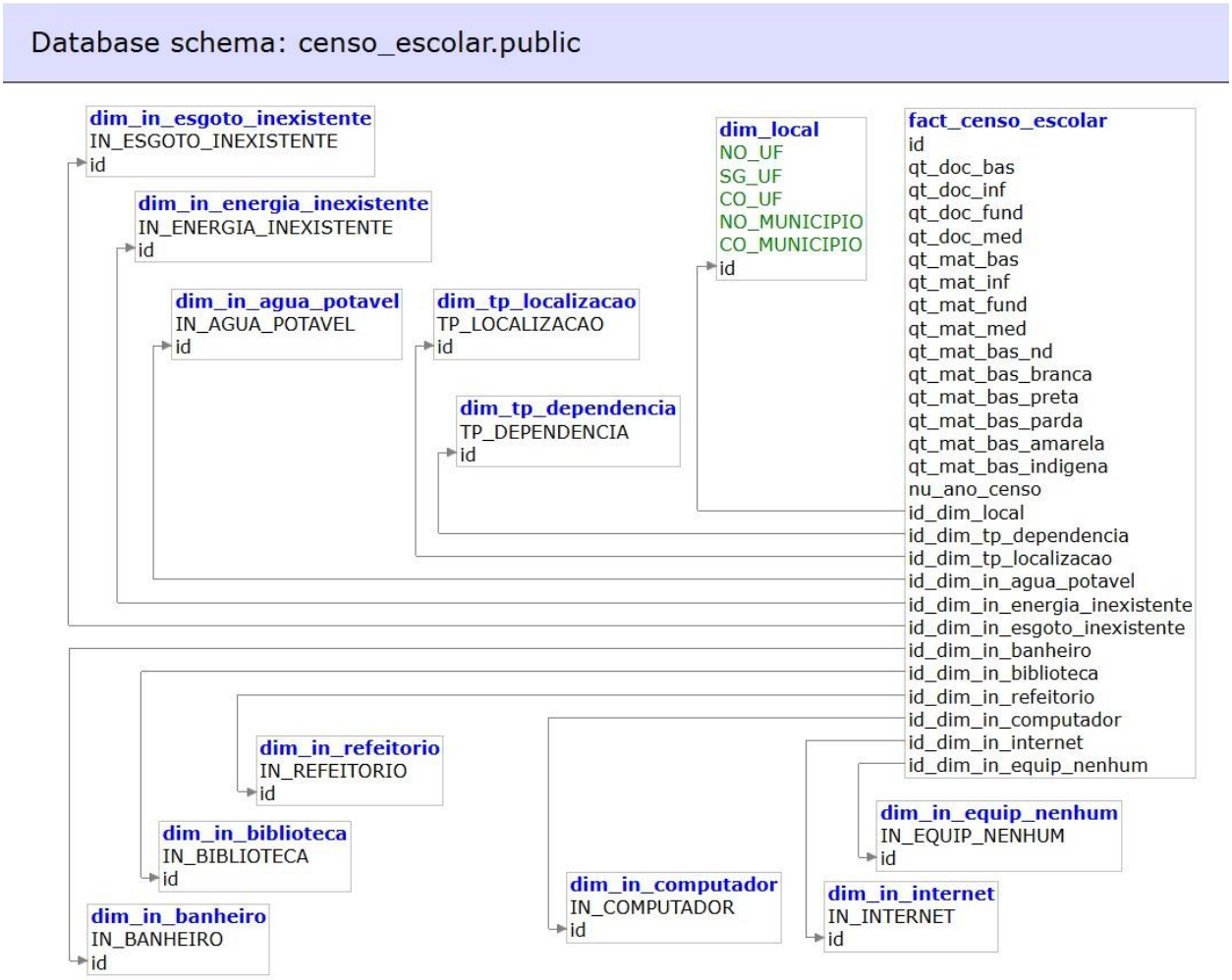

Figure 4.1 Star Schema

4.2 Hierarchy in Star Schema

4.2.1 Time Hierarchy

The time hierarchy focuses on the manual tracking of the dataset, using nu_ano_censo attribute directly in the fact_censo_escolar fact table. Since the school census data is collected and released annually, the single level represents the standard academic year cycle. It supports vertical trend analysis, allowing users to track progress, enrolment growth, and infrastructure changes for several consecutive years.

Academic Census Year (nu_ano_censo)

4.2.2 Location Hierarchy

The location hierarchy is built directly from the dim_local dimension table to support geographical analysis. The hierarchy begins at the country level (Brazil), then breaking down into specific state (NO_UF/SG_UF) and finally drilling down to the municipality level (NO_MUNICIPIO). This structured approach allows users to perform macro-level analysis at the national or state/province level while also supporting micro-drilling to examine key performance indicators (KPIs) for education and the status of school infrastructure within individual cities.

Country (Brazil) → State(NO_UF/SG_UF)→ Municipality/City (NO_MUNICIPIO)

4.3 Dimension Table Implementation

These dimension tables from the basis of the star schema, which contain the categories and attributes of each entity in the study. Table 4.1 below shows a 12-dimensional table and their respective row quantities and descriptions, which are implemented in our data model.

Dimension Table	Description	Rows
dim_local	All unique Brazilian state and municipality combinations. Supports the location hierarchy.	~5570
dim_tp_dependencia	Administrative reliance of the school (Federal, State, Municipal, Private)	4
dim_tp_localizacao	Geographic location zone type (Urban, Rural)	2
dim_in_banheiro	Availability of restrooms in the school: Yes/No	2
dim_in_internet	Availability of internet access: Yes/No	2

dim_in_agua_potavel	Availability of potable/drinking water: Yes/No	2
dim_in_biblioteca	Availability of a school library: Yes/No	2
dim_in_refeitorio	Availability of a cafeteria/refectory: Yes/No	2
dim_in_computador	Availability of computers for student use: Yes/No	2
dim_in_energia_inexistente	Indicator for total lack of electricity: Yes/No	2
dim_in_esgoto_inexistente	Indicator for total lack of sewage/sanitation system: Yes/No	2
dim_in_equip_nenhum	Indicator for schools having no educational equipment: Yes/No	2

The dim_local table contains about 5570 rows because it stores a unique combination of Brazilian states and cities which means that the number of unique locations combinations is high. In contrast, each facility infrastructure dimension table like dim_in_banheiro, dim_in_internet contains only 2 lines, indicating binary value, yes/no to determine whether the school has the facility. Dim_tp_dependencia includes 4 lines representing 4 types of school management, and dim_tp_localizacao includes 2 lines for urban and rural classification. These low-base dimensions allow high-speed filtering and quick connections between fact tables and dimension tables during dashboard rendering.

5.0 EVALUATION

5.1 Dashboard evaluation

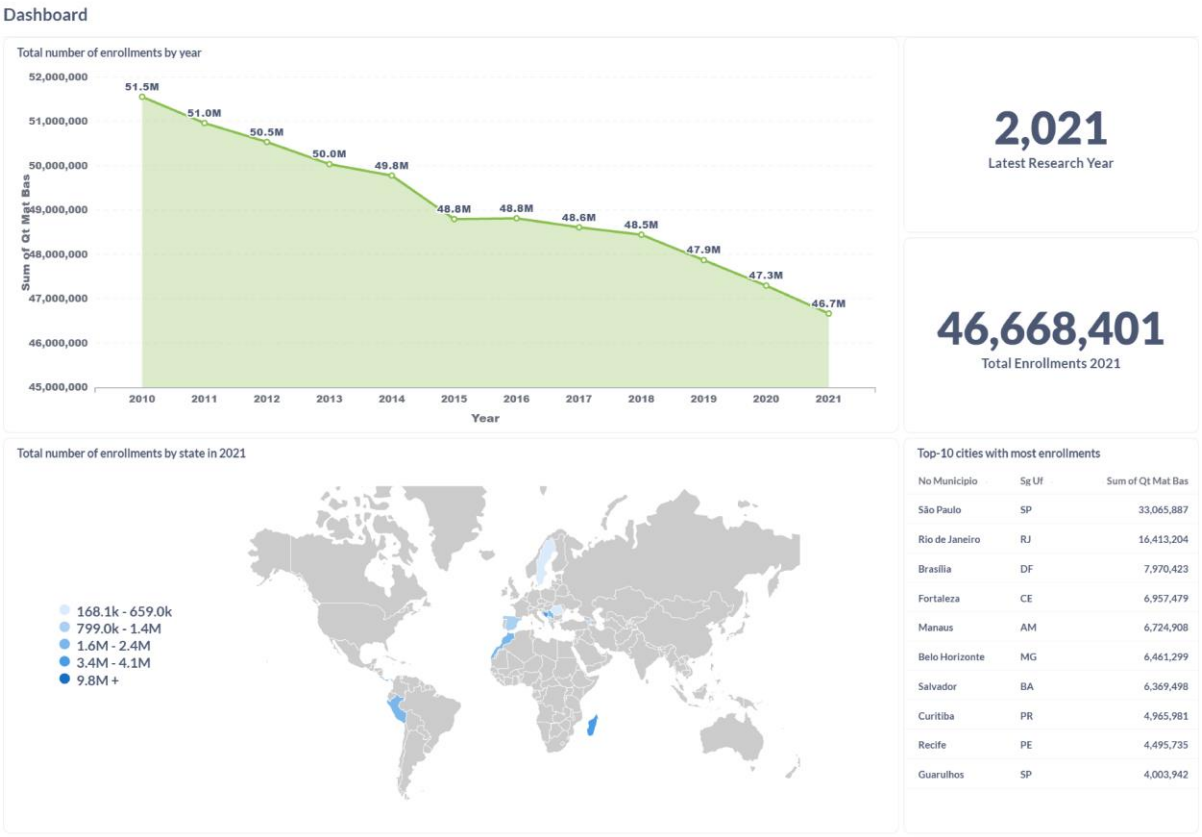

Figure 5.1 Dashboard in Metabase

The dashboard shown in Figure 5.1 was constructed in Metabase to evaluate the macro trend of the Brazilian basic education dataset on the pillars of time and geography. The upper side of the dashboard tracked the historical enrollment trends from 2010 to 2020, showing a continuous downward trend and finally stabilizing the baseline of 46.7M active registrations, highlighting the macro population transformation of Brazil's stable birth rate. At the same time, the bottom part of the dashboard evaluates the macro distribution of educational demand and identifying São Paulo as the final infrastructure anomaly with cumulative enrolment, 33,065,887. Next will be Rio de Janeiro which has 16,413,204 people and 7,970,423 for Brasília. The number of enrollments for the remaining city (Fortaleza, Manaus, Belo Horizonte, Salvador, Curitiba, Recife, Guarulhos) is between 7,970,423 - 4,003,942.

5.2 Limitation

One of the main limitations encountered in this tutorial is the mismatch between the tutorial environment and our implementation environment. This tutorial is designed for Linux-based systems, but when we learn about this tutorial, we use software on Windows such as Google Colab and Jupyter Notebook, which requires additional configuration steps and increases the complexity of the implementation. In addition, the temporary runtime of Colab means that all files, tables and session statuses are not saved, so the entire workflow must be re-executing every time the session is restarted, thus reducing efficiency. In addition, other tools such as PostgreSQL and Hadoop must be installed and configured to support data storage and processing, which may add additional complexity to the overall system integration. Besides, the dataset used in the tutorial also contained incomplete records and null values, which may affect the accuracy and reliability of the analysis results.

5.3 Improvement

In order to enhance the current architecture, the model can be improved by converting the traditional Star Schema into a hybrid medal architecture in bronze, silver and gold layer using the Data Lakehouse model to better manage future historical census data updates. In addition, enriching the dim_local dimension table with the official IBGE geospatial coordinates (latitude and longitude) will allow stakeholders to generate interactive geothermal maps in the dashboard, surpassing flat text analysis. Finally, implementing an incremental loading strategy based on the nu_ano_censo attribute and standardizing Portuguese source field names (such as NO_UF and SG_UF) into English business terms will significantly optimize the performance of the data warehouse, reduce the cost of cloud computing, and improve the global availability of self-service BI reports.

6.0 REFLECTION

Chua Jia Lin

This tutorial provided valuable practical experience in using Apache Spark, Docker, PostgreSQL, and Metabases for building a complete ETL pipeline and performing data analytics. Besides, it also enabled practical exposure with real-world data engineering tools and workflows, especially in handling large-scale datasets and preparing the data for analytical reporting.

A key learning from this tutorial was the implementation of a star schema, including the creation of dimension and fact tables using Python. This contributed a lot to the understanding of multidimensional data modeling.

However, few issues were encountered during the tutorial, such as environment configuration problems, difficulties in loading dimension tables into PostgreSQL, and challenges in configuring referential integrity between dimension and fact tables. Apart from these challenges, this tutorial enhanced the problem-solving skills in debugging ETL pipelines and strengthened the understanding of end-to-end data engineering processes.

Cheryl Cheong Kah Voon

Through this tutorial, I learned how to use Apache Spark, Docker, and Metabase to create ETL pipelines. During the setup and implementation process, I faced several problems, especially when configuring Apache Spark and solving compatibility problems related to Java and PySpark. These challenges require additional troubleshooting and research before the pipeline can be successfully operated.

Overall, this project has improved my understanding of ETL workflow, data conversion, and multidimensional modelling. I also have a better understanding of fact tables, dimension tables, measures, and hierarchy used in business intelligence systems. This tutorial has improved my skills and problem-solving skills in data engineering.

Elijah She Yu Sheng

Through this tutorial, I learned practical experience in understanding how Apache Spark can be applied to process large-scale datasets. My main responsibility was to complete the Business Understanding and Data Understanding sections. From the business perspective, I learned how to

identify the main problem of the project, which is that the Censo Escolar dataset is large, separated into multiple yearly CSV files, and not suitable for direct analysis without a proper data engineering workflow. This helped me understand the importance of connecting technical implementation with real analytical needs.

7.0 APPENDIX

download_censo_escolar.py

```
import os
import requests
import zipfile
import shutil
import urllib3
import time
import requests

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

YEARS = [str(i) for i in range(2010, 2022)]

BASE_URL = "https://download.inep.gov.br/dados_abertos/microdados_censo_escolar_{}.zip"

DATA_DIR = "./data"

os.makedirs(DATA_DIR, exist_ok=True)


def download_file(url, path, retries=5):
    for attempt in range(retries):
        try:
            print(f"Downloading {url} (attempt {attempt+1})")

            with requests.get(url, stream=True, timeout=60, verify=False) as r:
                r.raise_for_status()

                with open(path, "wb") as f:
                    for chunk in r.iter_content(chunk_size=1024 * 1024):  # 1MB chunks
                        if chunk:
                            f.write(chunk)

            print("Download complete ✓")
            return
        except Exception as e:
            print(f"Failed attempt {attempt+1}: {e}")
            time.sleep(5)

    raise Exception("Download failed after multiple retries")


for year in YEARS:
    zip_path = os.path.join(DATA_DIR, f"{year}.zip")
    extract_path = os.path.join(DATA_DIR, year)

    url = BASE_URL.format(year)

    # 1. Download
    if os.path.exists(zip_path):
        print(f"{zip_path} already exists, skipping download")
        continue

    download_file(url, zip_path)

    # 2. Extract
    print(f"Extracting {zip_path}")
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(extract_path)

    # 3. Find CSV file
    csv_file = None
    for root, dirs, files in os.walk(extract_path):
        for file in files:
            if file.lower().endswith(".csv"):
                csv_file = os.path.join(root, file)
                break

    # 4. Move CSV to main folder
    if csv_file:
        final_path = os.path.join(DATA_DIR, f"{year}.csv")
        shutil.move(csv_file, final_path)
        print(f"Saved: {final_path}")

    # 5. Cleanup
    shutil.rmtree(extract_path)
    os.remove(zip_path)

print("DONE ✓")
```

etl.py

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from datetime import datetime
from itertools import chain
import psycopg2

# ------------------------------------------------
# Test psycopg2
# ------------------------------------------------
try:
    import psycopg2
    print("psycopg2 imported OK")
except ImportError as e:
    print(f"psycopg2 import FAILED: {e}")

# ------------------------------------------------
# Spark Session
# ------------------------------------------------
spark = (
    SparkSession.builder
    .appName("ETL - Censo Escolar")
    .config(
        "spark.jars",
        r"C:\spark\jars\postgresql-42.7.3.jar"   # <-- CHANGE THIS
    )
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

# ------------------------------------------------
# PostgreSQL config
# ------------------------------------------------
PG_HOST = "postgres"
PG_PORT = 5432
PG_DB = "censo_escolar"
PG_USER = "censo"
PG_PASSWORD = "123"

PG_URL = f"jdbc:postgresql://{PG_HOST}:{PG_PORT}/{PG_DB}"

POSTGRES_CONFIG = {
    "url": PG_URL,
    "properties": {
        "user": PG_USER,
        "password": PG_PASSWORD,
        "driver": "org.postgresql.Driver"
    }
}

# ------------------------------------------------
# PostgreSQL helper
# ------------------------------------------------
def get_connection():
    return psycopg2.connect(
        host=PG_HOST,
        port=PG_PORT,
        database=PG_DB,
        user=PG_USER,
        password=PG_PASSWORD
    )

# ------------------------------------------------
# Load parquet
# ------------------------------------------------
print("Loading parquet...")

data = spark.read.parquet(
    "/app/data/censo_escolar.parquet"   # <-- CHANGE THIS
)

print(f"Total rows: {data.count()}")
```

```python
# ----------------------------------------------------
# Dimension tables
# ----------------------------------------------------
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM"
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL": {
        "fields": [
            {"field": "NO_UF", "type": "string"},
            {"field": "SG_UF", "type": "string"},
            {"field": "CO_UF", "type": "string"},
            {"field": "NO_MUNICIPIO", "type": "string"},
            {"field": "CO_MUNICIPIO", "type": "string"}
        ]
    }
}

DIMENSION_TABLES_CONFIG.update(
    {
        "DIM_" + dimension.upper(): {
            "fields": [
                {"field": dimension, "type": "integer"}
            ]
        }
        for dimension in INTEGER_DIMENSIONS
    }
)

# ----------------------------------------------------
# Write dimensions
# ----------------------------------------------------
for table_name, table_config in DIMENSION_TABLES_CONFIG.items():

    print(f"[{datetime.now()}] Writing {table_name}")

    dimension_df = (
        data
        .select(
            [
                F.col(field["field"])
                .cast(field["type"])
                .alias(field["field"])
                for field in table_config["fields"]
            ]
        )
        .distinct()
        .withColumn("id", F.monotonically_increasing_id())
    )
```

```
dimension_df.write.jdbc(
    url=POSTGRES_CONFIG["url"],
    table=table_name,
    mode="overwrite",
    properties=POSTGRES_CONFIG["properties"]
)

print(f"[{datetime.now()}] Wrote {table_name}")

# Add primary key
conn = get_connection()
cursor = conn.cursor()

try:
    cursor.execute(
        f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);"
    )
    conn.commit()

except Exception as e:
    print(f"PK already exists or error: {e}")
    conn.rollback()

finally:
    cursor.close()
    conn.close()

print(f"[{datetime.now()}] Done")
```

```
# ------------------------------------------------
# Facts table
# ------------------------------------------------
print("Creating facts table...")

FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"

FACT_COLUMNS = [
    "QT_DOC_BAS",
    "QT_DOC_INF",
    "QT_DOC_FUND",
    "QT_DOC_MED",
    "QT_MAT_BAS",
    "QT_MAT_INF",
    "QT_MAT_FUND",
    "QT_MAT_MED",
    "QT_MAT_BAS_ND",
    "QT_MAT_BAS_BRANCA",
    "QT_MAT_BAS_PRETA",
    "QT_MAT_BAS_PARDA",
    "QT_MAT_BAS_AMARELA",
    "QT_MAT_BAS_INDIGENA",
    "NU_ANO_CENSO"
]

FACT_CONFIG = {
    fact: {
        "fields": [
            {"field": fact, "type": "integer"}
        ]
    }
    for fact in FACT_COLUMNS
}

DIMENSION_ID_CONFIG = {
    table_name: [
        field["field"]
        for field in table_fields["fields"]
    ]
    for table_name, table_fields in DIMENSION_TABLES_CONFIG.items()
}

FACT_TABLE_ALL_COLUMNS_ORDERED = (
    FACT_COLUMNS
    + list(map(lambda col: "ID_" + col, DIMENSION_ID_CONFIG.keys()))
)
```

```
# ------------------------------------------------
# Create facts table
# ------------------------------------------------
comma_break_line = ",\n"

conn = get_connection()
cursor = conn.cursor()

try:

    # ------------------------------------------------
    # Create FACT table
    # ------------------------------------------------
    facts_table_sql = f"""
    CREATE TABLE IF NOT EXISTS {FACT_TABLE_NAME} (
        id SERIAL PRIMARY KEY,
        {
            comma_break_line.join(
                [f"{field} INTEGER" for field in FACT_COLUMNS]
                +
                [
                    f"ID_{dim_table} BIGINT"
                    for dim_table in DIMENSION_ID_CONFIG.keys()
                ]
            )
        }
    );
    """

    cursor.execute(facts_table_sql)
    conn.commit()

    print(f"[{datetime.now()}] Created facts table")
```

```
    # ------------------------------------------------
    # Add foreign keys
    # ------------------------------------------------
    for dim_table in DIMENSION_ID_CONFIG.keys():

        constraint_name = f"fk_{FACT_TABLE_NAME}_{dim_table}"

        fk_sql = f"""
        ALTER TABLE {FACT_TABLE_NAME}
        ADD CONSTRAINT {constraint_name}
        FOREIGN KEY (ID_{dim_table})
        REFERENCES {dim_table}(id);
        """

        try:

            cursor.execute(fk_sql)
            conn.commit()

            print(
                f"[{datetime.now()}] Added FK: "
                f"{FACT_TABLE_NAME} -> {dim_table}"
            )

        except Exception as fk_error:

            print(
                f"[{datetime.now()}] FK already exists "
                f"or error: {fk_error}"
            )

            conn.rollback()

except Exception as e:

    print(f"ERROR creating fact table: {e}")
    conn.rollback()

finally:

    cursor.close()
    conn.close()

# ------------------------------------------------
# Prepare facts data
# ------------------------------------------------
facts_data = data.select(
    [
        *chain(
            *DIMENSION_ID_CONFIG.values(),
            FACT_CONFIG.keys()
        )
    ]
)
```

```
# ------------------------------------------------
# Join dimensions
# ------------------------------------------------
for table_name, table_fields in DIMENSION_ID_CONFIG.items():

    dim_table = (
        spark.read.jdbc(
            url=POSTGRES_CONFIG["url"],
            table=table_name,
            properties=POSTGRES_CONFIG["properties"]
        )
        .withColumnRenamed("id", f"ID_{table_name}")
    )

    facts_data = (
        facts_data
        .join(
            dim_table,
            on=table_fields,
            how="left"
        )
        .drop(*table_fields)
    )

# ------------------------------------------------
# Write facts table
# ------------------------------------------------
try:

    (
        facts_data
        .select(*FACT_TABLE_ALL_COLUMNS_ORDERED)
        .write
        .jdbc(
            url=POSTGRES_CONFIG["url"],
            table=FACT_TABLE_NAME,
            mode="append",
            properties=POSTGRES_CONFIG["properties"]
        )
    )

    print("Facts table written successfully")

except Exception as e:
    print(f"ERROR in facts table: {e}")
```